PART VI — THE SELF-LEARNING LAYER

Chapter 29

Capturing Interactions: The Feedback Store

"A system cannot learn from an interaction it never wrote down."

Chapter 29 — Capturing Interactions: The Feedback Store

Chapter 28 drew the line between agentic behavior and genuine cross-run learning, and closed by naming the artifact every subsequent Part VI mechanism depends on: a durable, queryable record of what happened on every prior turn. This chapter builds that record — the feedback store — in the form this project originally shipped it: `interactions.jsonl`, a single flat, append-only file, plus the `FeedbackStore` class that reads and writes it.

This is a deliberate pedagogical choice worth stating up front. This project's feedback persistence layer was later migrated to MongoDB (ADR-046, covered in a later chapter) for concurrency and transactional reasons real production load exposed. But the flat-file design this chapter teaches is not a simplification invented for the book — it is the system's actual original architecture, and every downstream mechanism in Chapters 30 through 32 (blocklists, thumbdown lookups, distillation) was designed against exactly this file's shape before the storage engine underneath it ever changed.

29.1 What to log

> **Definition — Interaction record**
> A single durable entry capturing one query-answer exchange together with enough evidence — the retrieved chunks, the quality outcome, every query variant attempted — to support both failure-avoidance and success-distillation later, without needing to re-run the interaction.

The question "what to log" is really the question "what will a future mechanism need to read." Chapter 30's failure blocklist needs to know which query phrasings were tried and which of them retrieved nothing. Chapter 31's thumbdown lookup needs the user's own feedback text, the original query, and every chunk each variant surfaced. Chapter 32's distillation engine needs a verified answer and the source chunks that grounded it. A record designed without these three downstream consumers in mind ends up needing a second, incompatible log the moment any of them is built — which is exactly the trap a single, sufficiently rich schema from day one avoids.

The record this project settled on carries: a timestamp and request ID (identity), the query and answer text (content), a quality verdict of `OK`, `INSUFFICIENT`, or `USER_THUMBSDOWN` (outcome), the sources and retrieved chunks from both the document and learned-QA tracks (evidence), and the full list of query variants attempted during that run, not merely the one that ultimately succeeded (trajectory). Figure 29.1 lays out this anatomy directly.

The trajectory field deserves particular emphasis because it is the easiest to omit and the most valuable once Chapter 30 is built. A naive design logs only the final, successful query and its answer — after all, that pairing is what the user saw. But Chapter 30's entire blocklist mechanism depends on knowing about the queries that were tried and discarded along the way, not just the one that eventually worked. If those failed intermediate variants are never written down, the agent has no way to remember, on a future run, that a particular phrasing already led nowhere — it would have to rediscover that failure from scratch every single time the topic came up.

29.1B Sizing the record without bloating the ledger

A record rich enough to support three downstream consumers is not the same as a record that stores everything unboundedly. Full chunk content, repeated across every interaction that happened to retrieve it, would make the ledger grow far faster than the information it actually needs to preserve — the vector store already holds the authoritative full text, keyed by the same source identifiers a truncated preview can carry. This project's own chunk-storage helper caps stored previews to the first three chunks per track and truncates each to its `content` and `source` fields only, discarding embeddings, raw scores, and metadata the ledger does not need to fulfill its actual job.

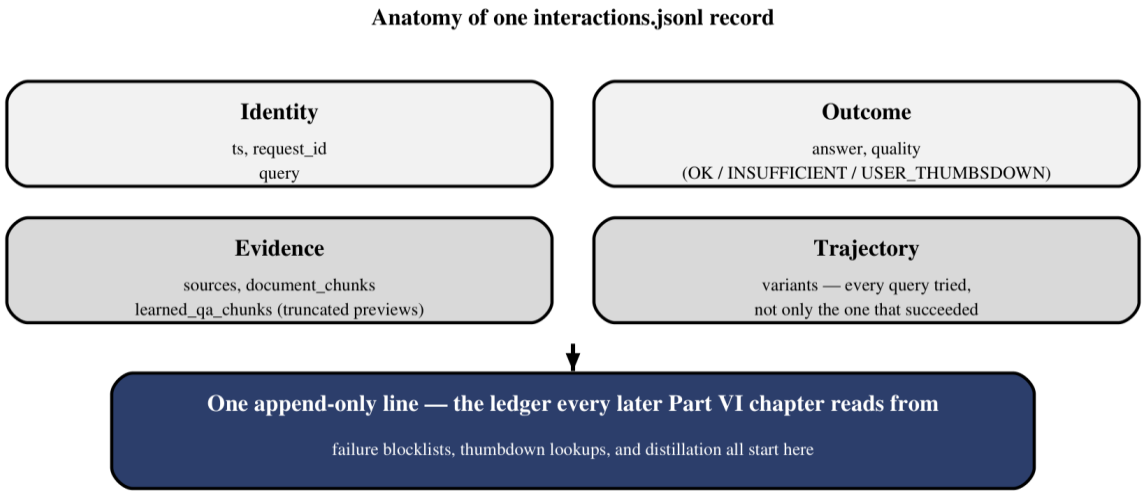

Figure 29.1 — Four field groups, one append-only record: identity, outcome, evidence, and trajectory.

29.2 JSONL as a lightweight ledger

JSON Lines — one complete, independent JSON object per line, rather than one JSON array wrapping every record — is a deliberately boring format choice, and the reasons it wins over the alternatives are entirely operational rather than aesthetic. A single JSON array requires reading, parsing, appending to, and re-serializing the *entire* file for every single write; at ten interactions that cost is invisible, at ten thousand it is a real bottleneck for what should be a cheap append. A JSONL file, by contrast, supports a true `open(path, "a")` append — write one line, flush, done — with no need to touch anything already on disk.

```
import json
from datetime import datetime, timezone

def log_interaction(path, query, answer, quality, sources, chunks,
variants):
    record = {
        "ts": datetime.now(timezone.utc).isoformat(),
        "query": query,
        "answer": answer,
        "quality": quality,                    # "OK" | "INSUFFICIENT" |
"USER_THUMBSDOWN"
        "sources": sources,
        "chunks": chunks,
        "variants": variants,
    }
    with open(path, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")
```

The append-only property has a second, quieter benefit this project's own bug history surfaced only after the later MongoDB migration: a flat JSONL file simply cannot raise a duplicate-key error on a retried write, because it has no concept of a unique key to violate — each append is just another line. The MongoDB successor gained a `request_id` uniqueness constraint specifically for data integrity, and then had to handle the exact `DuplicateKeyError` case a node retry could trigger — a failure mode the original flat file was structurally incapable of producing in the first place, though at the cost of the file format not being able to detect or prevent a true duplicate either.

> **Common pitfall — Reaching for a database before you need one**
> A JSONL file readable with a single `grep` or `pandas.read_json(lines=True)` call is easier to inspect, diff, and reason about during early development than a database schema migration. Reach for a real datastore when concurrency, transactions, or query patterns actually demand it — not preemptively, on the assumption that a file will not scale.

29.3 Implicit vs. explicit feedback

Every interaction produces feedback whether or not the user ever types a word about it. The JUDGE phase's `check_answer_quality()` call (Chapter 16.3) runs on every single turn, producing an `OK` or `INSUFFICIENT` verdict entirely automatically — this is implicit feedback, generated by the system's own quality gate, present for 100% of interactions regardless of user engagement. Explicit feedback — a user actually invoking the `bad` command and typing what was wrong — is a strictly rarer, richer signal: it requires the user to notice a problem, care enough to act on it, and articulate what was wrong in their own words.

Figure 29.2 lays these two paths side by side specifically to make their asymmetry visible: implicit feedback is high-volume and low-detail (a single word, `OK` or `INSUFFICIENT`), while explicit feedback is low-volume and high-detail (a full sentence describing exactly what the user expected instead). A system that only captured one of the two would be missing either the statistical coverage the first provides or the diagnostic depth the second provides — Chapter 32's distillation loop leans almost entirely on the first, while Chapter 31's thumbdown mechanism exists entirely because of the second.

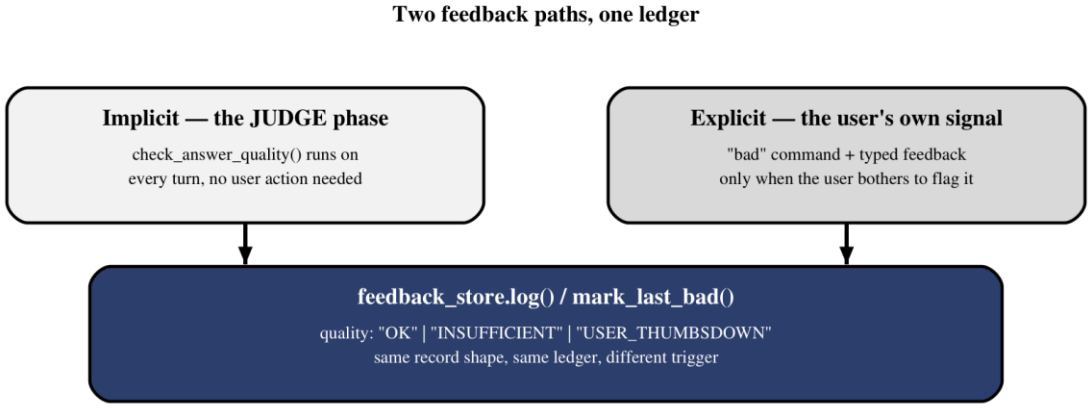

Figure 29.2 — Implicit feedback covers every turn automatically; explicit feedback is rarer but carries the user's own diagnosis of what went wrong.

Both signals write to the identical record shape and the identical ledger — `quality` simply takes a different value (`OK`/`INSUFFICIENT` from the judge, `USER_THUMBSDOWN` from the user) — which is precisely why Section 29.1's schema treats them as one unified concept rather than two separate logs. A distillation pipeline reading only `OK`-quality records (Chapter 32.1) does not need to know or care whether that verdict came from an automatic judge call; it only needs the guarantee that every `OK` record actually earned that label.

29.4 Building FeedbackStore

The `FeedbackStore` class wraps the raw file operations behind a small, stable interface: `log()` to append a new interaction, `load_all()` and `load_good()` to read records back (the latter filtered to `quality == "OK"`, exactly what Chapter 32's distillation loop needs), `count()` and `count_good()` for the running totals Chapter 35's `stats` command surfaces, and `mark_last_bad()` to retroactively flip the most recent record's quality when a thumbdown arrives after the fact.

```
class FeedbackStore:
    def __init__(self, path="data/feedback/interactions.jsonl"):
        self.path = path

    def log(self, query, answer, quality, sources, chunks, variants):
        ...  # append one JSONL line, as in Section 29.2

    def load_all(self) -> list[dict]:
        with open(self.path, encoding="utf-8") as f:
            return [json.loads(line) for line in f if line.strip()]

    def load_good(self, limit=None) -> list[dict]:
        good = [r for r in self.load_all() if r["quality"] == "OK"]
        return good[-limit:] if limit else good

    def count(self) -> int:
        return len(self.load_all())

    def count_good(self) -> int:
        return sum(1 for r in self.load_all() if r["quality"] == "OK")
```

This illustrative version re-reads the whole file on every call, which is the honest trade-off a flat-file store makes: simplicity and zero external dependencies, at the cost of O(n) reads that eventually motivate exactly the kind of indexed, queryable backend ADR-046 later introduced. `mark_last_bad()` is the one operation that complicates this simplicity most directly — it needs to find and rewrite one specific already-written line, which a pure-append format cannot do without reading and rewriting the entire file, foreshadowing the two-step-write consistency problem Chapter 31 and the later MongoDB migration both had to solve more carefully.

It is worth being explicit about why `load_good(limit=N)` matters as its own method rather than a slice applied by the caller. Chapter 32's distillation loop wants only the most recent N good interactions, not an arbitrary N — otherwise repeated distillation passes would keep re-processing the same oldest records forever while newer successes sat unlearned. Encoding "most recent good interactions" as the store's own responsibility, rather than trusting every caller to slice the list correctly, is the same single-source-of-truth discipline Chapter 27.6 argued for applied to a persistence layer instead of an in-memory counter.

29.5 Privacy, PII, and what NOT to log

A feedback store is, by design, a durable, growing record of exactly what users asked and exactly what the system told them — which makes it a natural place for personally identifying or sensitive information to accumulate quietly, entry by entry, unless the logging code actively guards against it. This project's own chunk-storage helper caps what gets persisted per chunk to content and source only, and truncates to a small preview length rather than storing full raw retrieval payloads — a deliberate minimization, not an oversight.

- Never log raw API keys, credentials, or authorization tokens, even if a user pastes one into a query by mistake — scan and redact before the write, not after.
- Truncate stored chunk previews rather than persisting full source documents a second time inside every interaction record — the vector store is already the source of truth for full content.
- Avoid logging free-text user feedback verbatim into any log stream with looser access controls than the feedback store itself; thumbdown text (Chapter 31) can contain more candid, sensitive detail than the original query.
- Treat `request_id` as an opaque correlation handle, not a place to smuggle session or user-identifying metadata that would otherwise need its own access-control review.
- Plan for deletion from the start — a flat-file ledger with no per-record deletion path makes a future "forget this user's data" request an all-or-nothing file rewrite rather than a targeted operation.

> **Common pitfall — Logging first, redacting later**
> Once a sensitive string is written to an append-only file, every backup, every log-shipping pipeline, and every downstream consumer that already read it has a copy. Filtering at write time is the only point where a single missed field does not become a permanent, distributed liability.

The chapters that follow this one all read from the ledger this chapter built. Chapter 30 asks what happens when the `variants` field records the same failing query phrasing turn after turn, and builds the blocklist that stops the agent from repeating it. Chapter 31 goes deeper into the `USER_THUMBSDOWN` records specifically — not just that a thumbdown happened, but what a system should actually do with the richer, harder-to-use signal a user's own written feedback provides.

One more property of this design is worth naming before moving on: nothing about the schema in Section 29.1 is specific to the flat-file backend that stores it. Every field — timestamp, query, answer, quality, evidence, variants — maps directly onto a MongoDB document's fields, which is precisely why ADR-046's later migration could describe itself as replacing the storage engine rather than redesigning the record. A schema designed around what downstream consumers need, independent of how it happens to be persisted, is what makes that kind of infrastructure swap a contained, mechanical change instead of a ledger-wide rewrite.

This separation of concerns — record shape as one decision, storage backend as an entirely separate one — is worth treating as a general design principle for any system expected to evolve past its first deployment. A project's very first feedback store rarely needs to survive concurrent writers or support transactional two-step updates; a JSONL file is honestly sufficient for that stage, and building anything heavier before the load justifies it is effort spent on a problem that does not yet exist. What does matter from day one is getting the record's *content* right, because a missing field is a data-loss problem no storage migration can retroactively fix — records written before a field existed simply never had it, no matter how sophisticated the database that later replaces the flat file becomes.